
Manual Técnico

Totito con Aprendizaje por Refuerzo

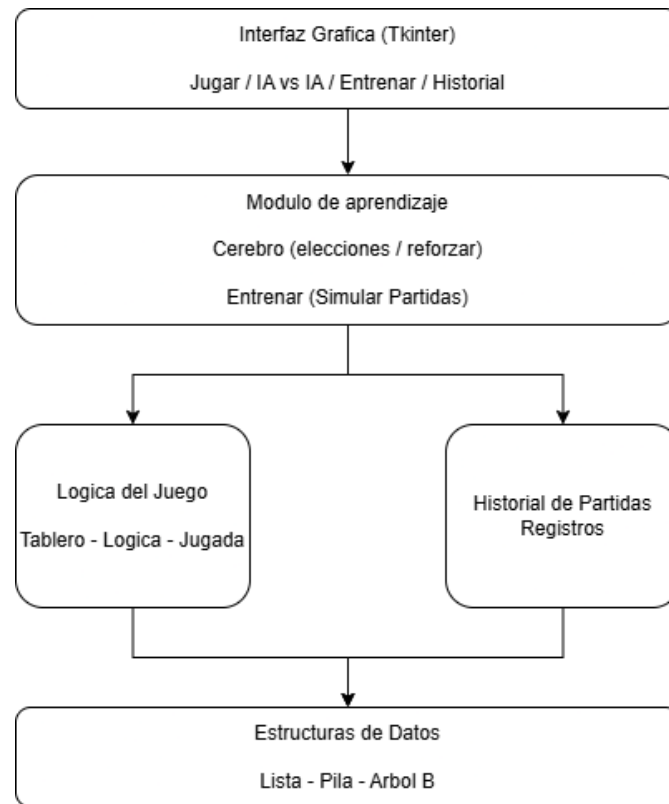
Universidad Mariano Gálvez — Facultad de Ingeniería en Sistemas

Programación III — Sección B — 2026

Integrante	Carné
Luis Emilio Sipac Maldonado	9490-24-5565
Herbert Rafael Lacan Hernandez	9490-24-21332

1. Esquema conceptual del sistema

El sistema está compuesto por cuatro capas que interactúan entre sí:



Flujo general: el usuario interactúa con la interfaz → la interfaz llama a Cerebro para elegir movimientos, Cerebro consulta su Arbol B. al terminar la partida, se actualizan los pesos en el árbol.

2. Arquitectura y componentes principales

2.1 Estructura de carpetas

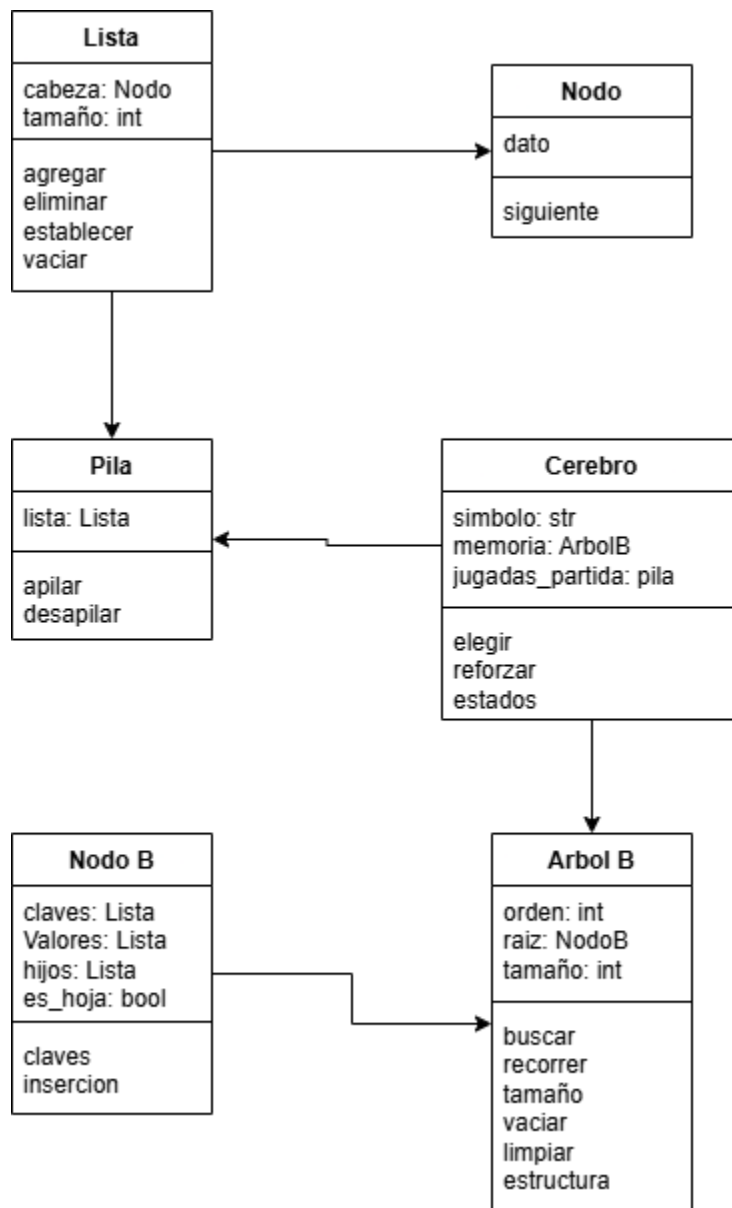
proyecto_final	
main.py	Punto de entrada
estructuras	
nodo.py	Nodo genérico (dato + siguiente)
lista.py	Lista enlazada simple
pila.py	Pila LIFO construida sobre Lista
arbol_b.py	Árbol B de orden configurable
juego	
jugada.py	Movimiento: fila, columna, peso
tablero.py	Tablero 3×3 (usa Lista internamente)
logica.py	ganador(), empate(), terminada()
aprendizaje	
cerebro.py	IA: Árbol B de estados con pesos
entrenador.py	Entrenamiento automático N partidas
historial	
registro.py	Una partida: id, ganador, jugadas
historial.py	Árbol B de registros históricos
visualizacion	
viz_partida.py	Grafo de una partida
viz_arbol.py	Grafo del Árbol B / camino IA
interfaz	
app.py	Ventana principal y todas las vistas

2.2 Dependencias externas

Paquete	Uso
tkinter	Interfaz gráfica (incluido en Python estándar)
graphviz	Renderizado de grafos en PNG (pip install graphviz)
Graphviz ejecutable	Motor dot para generar imágenes (graphviz.org)

3. Diagrama de clases

A continuación, se presenta el diagrama de clases del sistema con sus atributos y relaciones principales:



4. Descripción de clases

4.1 Nodo (estructuras/nodo.py)

Nodo genérico para construir estructuras enlazadas.

Atributo	Tipo	Descripción
dato	any	Dato almacenado en el nodo
siguiente	Nodo	Referencia al siguiente nodo (None si es el último)

4.2 Lista (estructuras/lista.py)

Lista enlazada simple. Cada elemento es un Nodo.

Atributo	Tipo	Descripción
cabeza	Nodo	Primer nodo de la lista
_tamano	int	Contador de elementos

Método	Descripción
agregar(dato)	Añade al final
agregar_inicio(dato)	Añade al inicio
insertar_en(indice, dato)	Inserta en posición específica
eliminar_en(indice)	Elimina y retorna el dato en esa posición
obtener(indice)	Retorna el dato sin eliminarlo
establecer(indice, dato)	Reemplaza el dato en esa posición
buscar(predicado)	Busca por función booleana

4.3 Pila (estructuras/pila.py)

Pila LIFO construida internamente sobre Lista. Se usa en Cerebro para registrar los movimientos de la partida actual mientras se juega.

Método	Descripción
apilar(dato)	Agrega al tope
desapilar()	Elimina y retorna el tope
tope()	Retorna el tope sin eliminar

vacía()	True si no tiene elementos
---------	----------------------------

4.4 NodoB (estructuras/arbol_b.py)

Nodo del Árbol B. Almacena claves, sus valores asociados y punteros a hijos.

Atributo	Tipo	Descripción
claves	Lista	Claves de búsqueda, ordenadas
valores	Lista	Datos asociados (paralelo a claves)
hijos	Lista	Punteros a nodos hijo (vacío si es hoja)
es_hoja	bool	True si no tiene hijos

4.5 ArbolB (estructuras/arbol_b.py)

Árbol B de orden configurable.

Atributo	Tipo	Descripción
orden	int	Máximo de claves por nodo antes de dividirse
raíz	NodoB	Nodo raíz
_tamano	int	Número total de pares clave-valor

Método	Descripción
buscar(clave)	Busca en cualquier nivel; retorna valor o None
insertar(clave, valor)	Inserta o actualiza; divide si hay desbordamiento
recorrer()	Recorrido inorden → Lista de (clave, valor)
todos()	Recorrido inorden → Lista de valores
tamano()	Número de entradas
vacio()	True si no tiene entradas
limpiar()	Reinicia el árbol
estructura()	Lista para visualización

4.6 Jugada (juego/jugada.py)

Representa un movimiento posible en un estado del tablero, junto con su peso de aprendizaje.

Atributo	Tipo	Descripción
fila	int	Fila (0-2)

columna	int	Columna (0-2)
peso	float	Puntuación aprendida (inicia en 1.0)

4.7 Tablero (juego/tablero.py)

Tablero 3×3 representado internamente como una Lista de 9 elementos (espacios en blanco, 'X' u 'O').

Método	Descripción
colocar(f, c, simbolo)	Coloca una ficha si la celda está vacía
obtener(f, c)	Retorna el símbolo en esa celda
movimientos()	Lista de (fila, col) disponibles
clave()	String de 9 chars que representa el estado
copiar()	Retorna un tablero nuevo con el mismo estado
deshacer(f, c)	Limpia una celda
reiniciar()	Vacía todo el tablero

4.8 Cerebro (aprendizaje/cerebro.py)

Núcleo de la inteligencia artificial. Mantiene un Arbol B donde la clave es el estado del tablero (string de 9 caracteres) y el valor es una Lista de Jugada con sus pesos.

Atributo	Tipo	Descripción
simbolo	str	'X' o 'O'
_memoria	ArbolB	Estado → Lista de Jugadas con pesos
_jugadas_partida	Pila	Movimientos de la partida actual

Constantes de aprendizaje:

Constante	Valor	Significado
AJUSTE_GANAR	+1.0	Premio por ganar
AJUSTE_EMPATE	+0.3	Premio por empatar
AJUSTE_PERDER	-0.8	Penalización por perder
PESO_MINIMO	0.05	Peso mínimo que puede tener una jugada
GAMMA	0.9	Factor de descuento temporal

Método	Descripción
elegir(tablero, exploracion)	Elige el mejor movimiento (o uno aleatorio si explora)

reforzar(resultado)	Ajusta los pesos de la partida recién terminada
estados()	Número de estados conocidos en el árbol

4.9 Entrenador (aprendizaje/entrenador.py)

Simula N partidas completas entre dos cerebros con exploración del 30% para garantizar diversidad de estados visitados.

Método	Descripción
entrenar(cx, co, n, historial)	Simula n partidas; retorna estadísticas

Retorna un diccionario con: `x` (victorias), `o` (victorias), `empate`, `total`, `partidas` (Lista de Registro).

4.10 Registro (historial/registro.py)

Representa una partida guardada. El ID se asigna automáticamente y se incrementa con cada partida.

Atributo	Tipo	Descripción
id	int	Identificador único auto-incremental
ganador	str	'X', 'O' o None (empate)
jugadas	Lista	Secuencia de (fila, col, símbolo)

4.11 Historial (historial/historial.py)

Almacena todos los registros de partidas en un Arbol B. La clave de búsqueda es el ID entero del registro.

Método	Descripción
agregar(ganador, jugadas)	Crea un Registro y lo inserta en el árbol
obtener(id)	Busca un Registro por ID
todos()	Recorrido inorden de todos los registros
estadisticas()	Victorias, empates, promedio de jugadas

5. Estructuras de datos implementadas

5.1 Lista enlazada

Propósito: almacenamiento

Uso en el sistema:

- El tablero 3×3 está representado como una Lista de 9 elementos.
- Las jugadas posibles en un estado son una Lista de objetos Jugada.
- El historial de movimientos de una partida es una Lista de tuplas.
- Las claves, valores e hijos de cada NodoB son Listas.

Implementación: cada nodo tiene dato y siguiente. Las operaciones de inserción y eliminación en posición arbitraria recorren la lista hasta el índice deseado ($O(n)$).

5.2 Pila (LIFO)

Propósito: registrar los movimientos de la IA durante una partida.

Uso en el sistema: Cerebro._Jugadas partida guarda (clave_tablero, jugada) por cada movimiento que la IA realiza. Al terminar la partida, reforzar() desapila desde el tope (última jugada) hacia abajo (primera jugada), aplicando descuento decreciente.

Implementación: Lista la usa agregar_inicio para que el tope sea siempre el primero.

5.3 Árbol B

Propósito: búsqueda e inserción eficiente de estados del tablero (para la IA) y de registros de partidas (para el historial).

Justificación de uso: el árbol B permite búsqueda en cualquier nivel. El orden configurable permite controlar el tamaño de los nodos y el número de niveles del árbol.

Parámetros de división:

- Un nodo se divide cuando acumula orden claves (desbordamiento).
- Índice mediano: mitad = num_claves // 2
- La clave en mitad sube al padre; las anteriores quedan en el nodo izquierdo y las posteriores en el nodo derecho.

Dos instancias distintas en el proyecto:

Instancia	Clave	Valor
Cerebro._memoria	Estado del tablero (9 chars)	Lista de Jugadas con pesos
Historial._arbol	ID de partida (entero)	Objeto Registro

6. Lógica del aprendizaje implementado

6.1 Inicialización de estados

Al iniciar la aplicación, poblar_todos_estados(cerebro_x, cerebro_o) recorre los estados válidos del totito e inserta cada uno en el árbol del cerebro correspondiente con todas sus jugadas posibles a peso 1.

6.2 Refuerzo con descuento temporal

Al terminar una partida se llama reforzar(resultado). El sistema aplica **descuento temporal** (factor GAMMA = 0.9): las jugadas más recientes reciben el mayor ajuste porque son las más decisivas para el resultado.

Ejemplo con una derrota (3 jugadas de O):

Jugada	Orden	Descuento	Ajuste
O juega (1,1) — 1ª	más antigua	$0.9^2 = 0.81$	$-0.8 \times 0.81 = -0.65$
O juega (0,2) — 2ª	intermedia	$0.9^1 = 0.90$	$-0.8 \times 0.90 = -0.72$
O juega (2,0) — 3ª	más reciente	1.00	$-0.8 \times 1.00 = -0.80$

La última jugada (la más cercana a la derrota) recibe la mayor penalización. Esto enseña a la IA cuáles fueron las decisiones más críticas.

6.3 Convergencia

Con 500 partidas de entrenamiento (exploración = 0.3) ambos cerebros convergen a juego óptimo: todos los juegos entre ellos terminan en empate, que es el resultado perfecto del totito cuando ambos jugadores juegan sin errores.

7. Descripción del aprendizaje supervisado y automático

Aprendizaje por refuerzo

El sistema implementa un agente de aprendizaje por refuerzo basado en tabla de valores (Q-table simplificada). Cada par (estado_tablero, jugada) tiene un valor Q representado por el peso de la Jugada.

Concepto RL	Equivalente en el sistema
Estado (s)	tablero.clave() — string de 9 chars
Acción (a)	Objeto Jugada (fila, columna)
Valor Q(s,a)	jugada.peso
Recompensa (r)	AJUSTE_GANAR / EMPATE / PERDER
Descuento (γ)	GAMMA = 0.9
Política de explotación	elegir jugada con mayor peso
Política de exploración	elegir jugada aleatoria (epsilon-greedy)
